From **OT** to **OLE** with Almost-Linear Communication

Geoffroy Couteau, Naman Kumar — TCC 2026, New York City

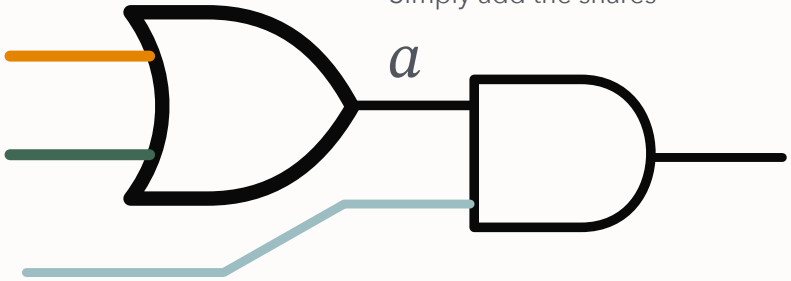


Reminder: How to do MPC?



$\langle x \rangle + \langle y \rangle = \langle a \rangle$

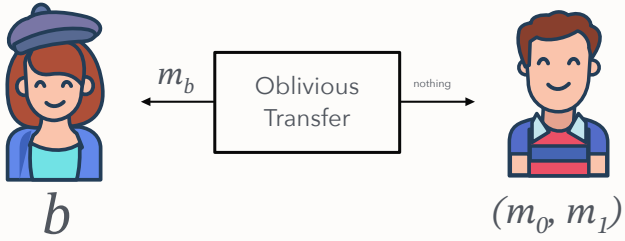
- **XOR Gate**
  - Local computation
  - Simply add the shares



- **AND Gate**
  - Need communication
  - Two instances of 'oblivious transfer'



$\langle x \rangle + \langle y \rangle = \langle a \rangle$

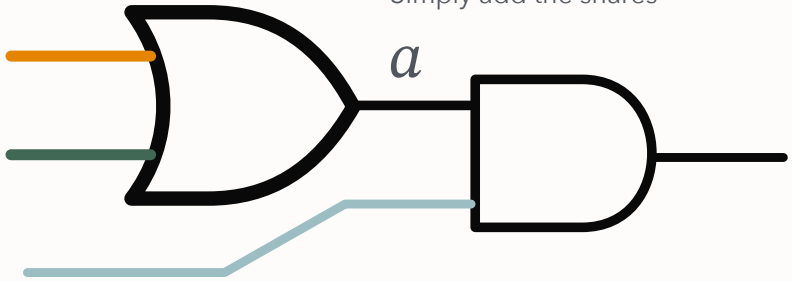


Reminder: How to do MPC?



$\langle x \rangle + \langle y \rangle = \langle a \rangle$

- **XOR Gate**
  - Local computation
  - Simply add the shares



- **AND Gate**
  - Need communication
  - Two instances of 'oblivious transfer'

THE GMW WAY!



$\langle x \rangle + \langle y \rangle = \langle a \rangle$



b

m_b

Oblivious Transfer

nothing



(m_0, m_1)

Reminder: How to do MPC?



$\langle x \rangle + \langle y \rangle = \langle a \rangle$

- **XOR Gate**
  - Local computation
  - Simply add the shares

THE GMW WAY!



- **AND Gate**
  - Need communication
  - Two instances of 'oblivious transfer'



$\langle x \rangle + \langle y \rangle = \langle a \rangle$





b

m_b

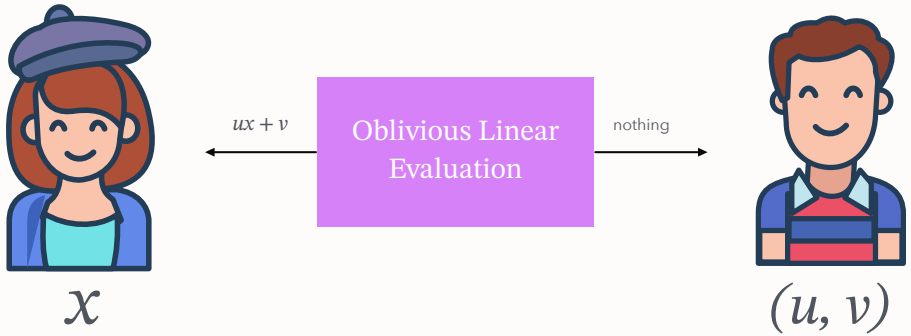
Oblivious Transfer

nothing

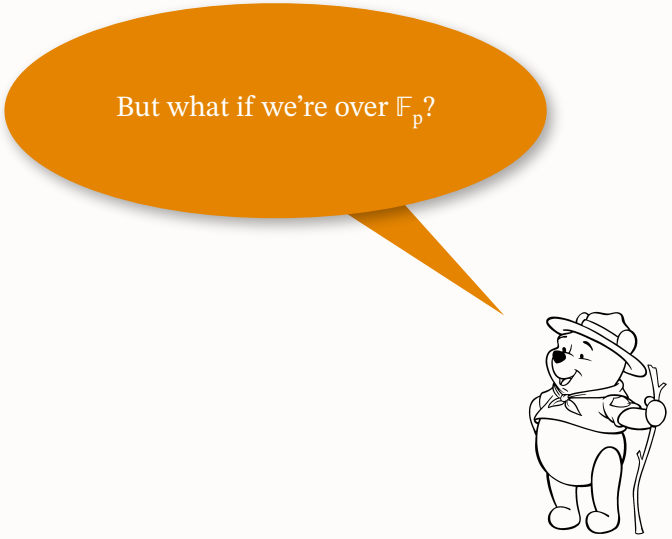


(m_0, m_1)

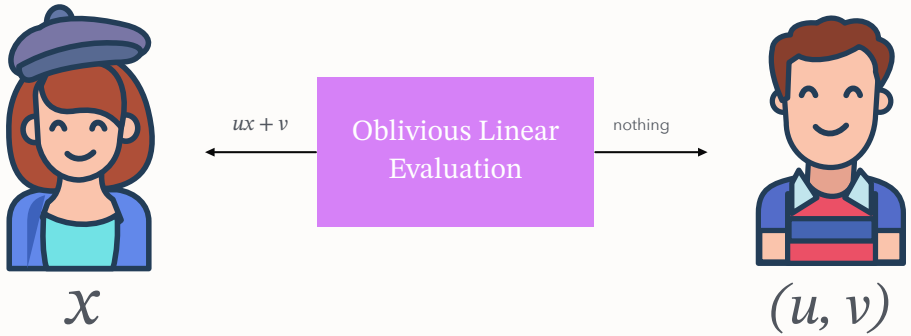
Reminder: How to do MPC?



- MPC over Arithmetic Circuits
- Used an an intermediary in, ex. threshold ECDSA
- Used for OPRF, PSI etc.



Reminder: How to do MPC?



- MPC over Arithmetic Circuits
- Used an an intermediary in, ex. threshold ECDSA
- Used for OPRF, PSI etc.

What is the Communication Complexity of OLE?

But what if we're over $\mathbb{F}_p$?



Review: OLE from OT

n-bit OLE



(a, b)



$x = 11010$

Review: OLE from OT

n-bit OLE



(a, b)

$$b = \sum_{i=1}^{5} b_i \pmod{n}$$



$x = 11010$

Review: OLE from OT

n-bit OLE



(a, b)

$b = \sum_{i=1}^{5} b_i \pmod{n}$

$(b_1, b_1 + a)$

$(b_2, b_2 + 2a)$

$(b_3, b_3 + 4a)$

$(b_4, b_4 + 8a)$

$(b_m, b_m + 16a)$

0

1

0

1

1



$x = 11010$

Review: OLE from OT

n-bit OLE



(a, b)

$b = \sum_{i=1}^{5} b_i \pmod{n}$

$(b_1, b_1 + a)$ → OT ← 0, b_1

$(b_2, b_2 + 2a)$ → OT ← 1, $b_2 + 2a$

$(b_3, b_3 + 4a)$ → OT ← 0, b_3

$(b_4, b_4 + 8a)$ → OT ← 1, $b_4 + 8a$

$(b_m, b_m + 16a)$ → OT ← 1, $b_m + 16a$



$x = 11010$

$b_1 + b_2 + b_3 + b_4 + b_5$
$0 + 2a + 0 + 8a + 16a$ $= b + (11010)a$

Review: OLE from OT

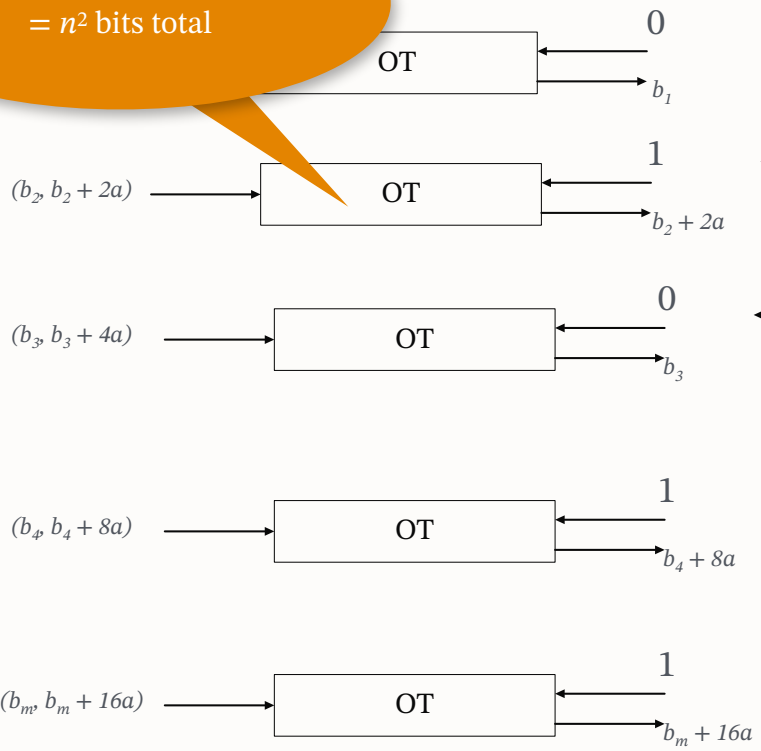
n-bit OLE

n bits of communication × *n* OT
= n^2 bits total



(a, b)

$b = \sum_{i=1}^{5} b_i \pmod{n}$





$x = 11010$

$b_1 + b_2 + b_3 + b_4 + b_5$
$0 + 2a + 0 + 8a + 16a$ $= b + (11010)a$

[DHIM25] - Using CRT

OLE over a ring $\mathbb{Z}_k$ where k is a smooth integer

$$k = 3 \cdot 5 \cdot \ldots \cdot k_m$$

$$m \approx \frac{\log k}{\log \log k}$$



(a, b)

$a = (a_1, \ldots, a_m)$
$b = (b_1, \ldots, b_m)$



$x = (x_1, \ldots, x_m)$

[DHIM25] - Using CRT

OLE over a ring $\mathbb{Z}_k$ where k is a smooth integer

$$k = 3 \cdot 5 \cdot \ldots \cdot k_m$$

$$m \approx \frac{\log k}{\log \log k}$$



(a, b)

$a = (a_1, \ldots, a_m)$
$b = (b_1, \ldots, b_m)$

(a_1, b_1)

(a_2, b_2)

(a_3, b_3)

(a_m, b_m)

x_1

x_2

x_3

$\vdots$

x_m



$x = (x_1, \ldots, x_m)$

$ax + b = (a_1x_1 + b_1, \ldots, a_mx_m + b_m)$

[DHIM25] - Using CRT

OLE over a ring $\mathbb{Z}_k$ where k is a smooth integer

$k = 3 \cdot 5 \cdot \ldots \cdot k_m$

$m \approx \frac{\log k}{\log \log k}$



(a, b)

$a = (a_1, \ldots, a_m)$

$b = (b_1, \ldots, b_m)$

(a_1, b_1) → Gilboa ← x_1, $a_1x_1 + b_1$

(a_2, b_2) → Gilboa ← x_2, $a_2x_2 + b_2$

(a_3, b_3) → Gilboa ← x_3, $a_3x_3 + b_3$

⋮

(a_m, b_m) → Gilboa ← x_m, $a_mx_m + b_m$



$x = (x_1, \ldots, x_m)$

$ax + b = (a_1x_1 + b_1, \ldots, a_mx_m + b_m)$

[DHIM25] - Using CRT

OLE over a ring $\mathbb{Z}_k$ where k is a smooth integer

$k = 3 \cdot 5 \cdot \ldots \cdot k_m$

$m \approx \frac{\log k}{\log \log k}$



(a, b)

$a = (a_1, \ldots, a_m)$

$b = (b_1, \ldots, b_m)$

$(\log \log k)^2$ bits of communication

(a_1, b_1) → Gilboa ← x_1 ; $a_1x_1 + b_1$

(a_2, b_2) → Gilboa ← x_2 ; $a_2x_2 + b_2$

(a_3, b_3) → Gilboa ← x_3 ; $a_3x_3 + b_3$

⋮

(a_m, b_m) → Gilboa ← x_m ; $a_mx_m + b_m$



$x = (x_1, \ldots, x_m)$

$ax + b = (a_1x_1 + b_1, \ldots, a_mx_m + b_m)$

[DHIM25] - Using CRT

OLE over a ring $\mathbb{Z}_k$ where k is a smooth integer

$$k = 3 \cdot 5 \cdot \ldots \cdot k_m$$

$$m \approx \frac{\log k}{\log \log k}$$



(a, b)

$a = (a_1, \ldots, a_m)$

$b = (b_1, \ldots, b_m)$

(a_1, b_1)

(a_2, b_2)

(a_3, b_3)

(a_m, b_m)

Gilboa

Gilboa

x_1

x_m

$a_m x_m + b_m$

$(\log \log k)^2$ bits of communication

Communication =

$$(\log \log k)^2 \times \frac{\log k}{\log \log k} = \log k \log \log k$$

= O(n log n) bits! (if k is of bitlength n)



$x = (x_1, \ldots, x_m)$

$$ax + b = (a_1 x_1 + b_1, \ldots, a_m x_m + b_m)$$

What about arbitrary rings ?

OLE over $\mathbb{Z}_q$

- Idea: Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

- How?

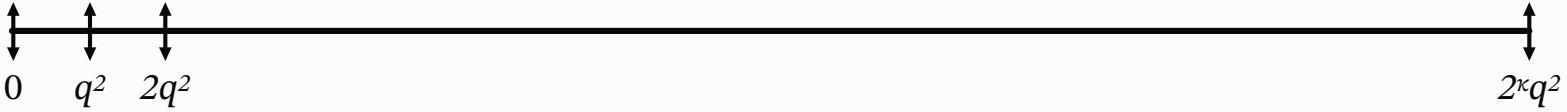
What about arbitrary rings ?

OLE over $\mathbb{Z}_q$

- Idea: Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

- How?

1. Choose a statistical security parameter κ
2. Work in the domain $[0,(2^\kappa - 1) \cdot q^2)$



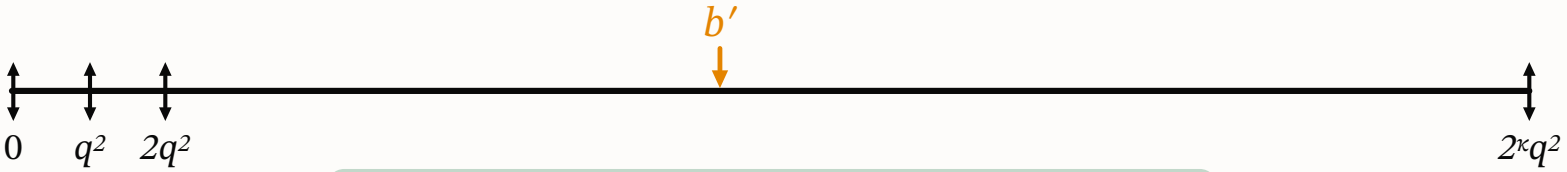
What about arbitrary rings ?

OLE over $\mathbb{Z}_q$

- Idea: Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

- How?

1. Choose a statistical security parameter κ
2. Work in the domain $[0,(2^\kappa - 1) \cdot q^2)$
3. Receiver chooses a random $b' = b \pmod q$
4. Work over $n > 2^\kappa \cdot q^2$



Think of this as an OLE over $\mathbb{Z}$!

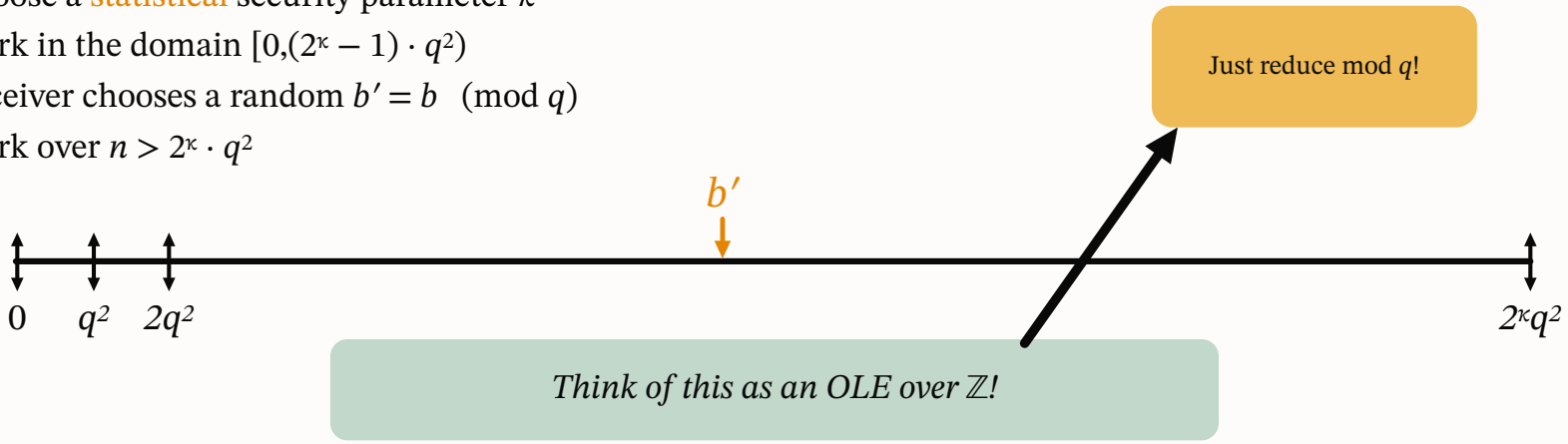
What about arbitrary rings ?

OLE over $\mathbb{Z}_q$

- Idea: Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

- How?

1. Choose a statistical security parameter κ
2. Work in the domain $[0,(2^\kappa - 1) \cdot q^2)$
3. Receiver chooses a random $b' = b \pmod q$
4. Work over $n > 2^\kappa \cdot q^2$



What about arbitrary rings ?

OLE over $\mathbb{Z}_q$

- Idea: Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

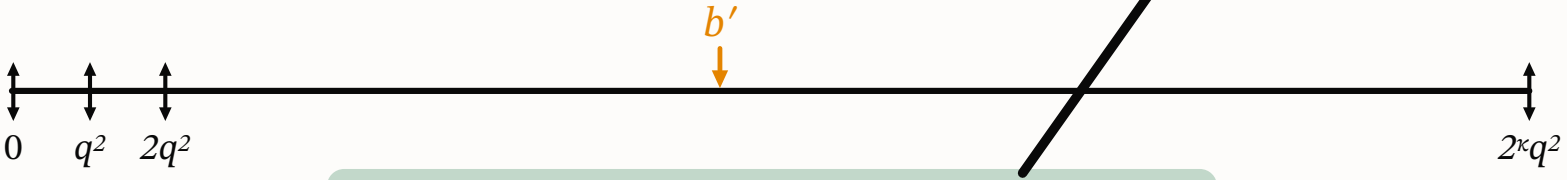
- How?

1. Choose a statistical security parameter κ
2. Work in the domain $[0,(2^\kappa - 1) \cdot q^2)$
3. Receiver chooses a random $b' = b \pmod q$
4. Work over $n > 2^\kappa \cdot q^2$

What is κ?

Jack, Yash: 80
Anu, Lance: 40
Damiano: 64,128

Just reduce mod q!



Think of this as an OLE over $\mathbb{Z}$*!*

The Idea: Why stop at one CRT?

$$\mathsf{OLE}_{\mathbb{Z}_n} = \mathsf{OLE}_{\mathbb{Z}_{n_1}} \times \underbrace{\mathsf{OLE}_{\mathbb{Z}_{n_2}}}_{\mathsf{OLE}_{\mathbb{Z}_{n_2}} = \mathsf{OLE}_{\mathbb{Z}_{n_{2,1}}} \times \mathsf{OLE}_{\mathbb{Z}_{n_{2,2}}} \times \cdots \times \mathsf{OLE}_{\mathbb{Z}_{n_{2,k2}}}} \times \cdots \times \mathsf{OLE}_{\mathbb{Z}_{n_k}}$$

} until everything turns constant = log* times!

Communication =

$$T(n) = \frac{n}{\log n} \times \mathrm{T}(\log n) + O(n)$$

$$= O(n \cdot \log^* n)!$$

The Idea: Why stop at one CRT?

$$\mathrm{OLE}_{\mathbb{Z}_n} = \mathrm{OLE}_{\mathbb{Z}_{n_1}} \times \underbrace{\mathrm{OLE}_{\mathbb{Z}_{n_2}}}_{\mathrm{OLE}_{\mathbb{Z}_{n_2}} = \mathrm{OLE}_{\mathbb{Z}_{n_{2,1}}} \times \mathrm{OLE}_{\mathbb{Z}_{n_{2,2}}} \times \cdots \times \mathrm{OLE}_{\mathbb{Z}_{n_{2,k2}}}} \times \cdots \times \mathrm{OLE}_{\mathbb{Z}_{n_k}}$$

} until everything turns constant = log* times!

Communication =

$$T(n) = \frac{n}{\log n} \times \mathrm{T}(\log n) + O(n)$$

$$= O(n \cdot \log^* n)!$$

The Problem...

There's a κ!

$$\mathrm{OLE}_{\mathbb{Z}_n} = \kappa \cdot \mathrm{OLE}_{\mathbb{Z}_{n_1}} \times \kappa \cdot \mathrm{OLE}_{\mathbb{Z}_{n_2}} \times \cdots \times \kappa \cdot \mathrm{OLE}_{\mathbb{Z}_{n_k}}$$

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!



(a, b)

$b - r$



x

$ax + r$

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!



(a, b)

$b - r$

$\langle ax + b \rangle_{\mathbb{Z}}$



x

$ax + r$

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!

If the share is of $ax + b$ then we can just reduce (mod q)...





(a, b)

$b - r$

$\langle ax + b \rangle_{\mathbb{Z}}$

x

$ax + r$

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!



If the share is of $ax + b$ then we can just reduce (mod q)...

What if it's of $ax + b + n$?



(a, b)

$b - r$

$\langle ax + b \rangle_{\mathbb{Z}}$

x

$ax + r$

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!



If the share is of $ax + b$ then we can just reduce (mod q)...

Then I'll change my share to $b - r - n$!

What if it's of $ax + b + n$?



(a, b)

$b - r$

$\langle ax + b \rangle_{\mathbb{Z}}$

x

$ax + r$

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!



(a, b)

$b - r$

$\langle y \rangle_{\mathbb{Z}}^{S}$

Secure Comparison

$Is\ \langle y \rangle_{\mathbb{Z}}^{R} + \langle y \rangle_{\mathbb{Z}}^{S} > n?$



x

$ax + r$

$\langle y \rangle_{\mathbb{Z}}^{R}$

New Modular Reduction

- Choose the first primorial $n > q$
- Run the protocol over $\mathbb{Z}_n$
- Reduce

Main idea: Instead of paying the overhead, reduce to one OT and one secure comparison!



(a, b)

$b - r$

$\langle y \rangle_{\mathbb{Z}}^{S}$

$\langle b \rangle^{S}$

Secure Comparison

$\langle b \rangle^{R}$

$Is\ \langle y \rangle_{\mathbb{Z}}^{R} + \langle y \rangle_{\mathbb{Z}}^{S} > n?$

$(\langle y \rangle_{\mathbb{Z}}^{S} - n\langle b \rangle^{S}, \langle y \rangle_{\mathbb{Z}}^{S} - n(1 - \langle b \rangle^{S}))$

OT

$\langle b \rangle^{R}$



x

$ax + r$

$\langle y \rangle_{\mathbb{Z}}^{R}$

Secure Comparison



Secure Comparison

$Is\ \langle y \rangle_{\mathbb{Z}}{}^{R} + \langle y \rangle_{\mathbb{Z}}{}^{S} > n?$



$$T(n) = O\left(\frac{n}{\log n}\right) \times \mathrm{T}(\log n) + O(?)$$

If $O(n \log n)$... you get $O(n \log n)$

If $O(n \log^* n)$... you get $O(n(\log^* n)^2)$

If $O(n)$... you get $O(n^{1+o(1)})$!

Secure Comparison



Secure Comparison

$Is\ \langle y \rangle_{\mathbb{Z}}{}^{R} + \langle y \rangle_{\mathbb{Z}}{}^{S} > n?$



$$T(n) = O\left(\frac{n}{\log n}\right) \times \mathrm{T}(\log n) + O(?)$$

If $O(n \log n)$... you get $O(n \log n)$

If $O(n \log^* n)$... you get $O(n(\log^* n)^2)$

If $O(n)$... you get $O(n^{1+o(1)})$!



Round complexity of protocol ≈
Round complexity of
Secure comparison!

Secure Comparison with Linear Communication and $O(log^* x)$ Rounds



Secure Comparison

$Is\ \langle y \rangle_{\mathbb{Z}}{}^{R} + \langle y \rangle_{\mathbb{Z}}{}^{S} > n?$



Secure Comparison with Linear Communication and $O(log^* x)$ Rounds



Secure Comparison

$Is\ \langle y \rangle_{\mathbb{Z}}{}^{R} + \langle y \rangle_{\mathbb{Z}}{}^{S} > n?$



Step 1: Equality Test





Secure Comparison with Linear Communication and $O(log^* x)$ Rounds

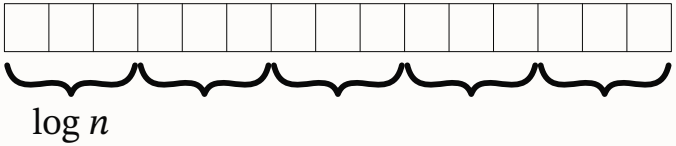


Secure Comparison

$Is\ \langle y \rangle_{\mathbb{Z}}{}^{R} + \langle y \rangle_{\mathbb{Z}}{}^{S} > n?$



Step 1: Equality Test





Secure Comparison with Linear Communication and $O(log^* x)$ Rounds

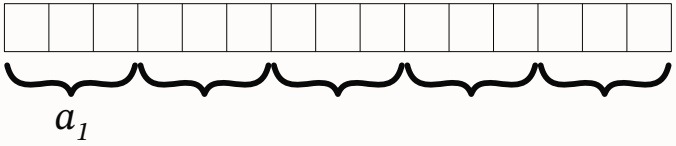


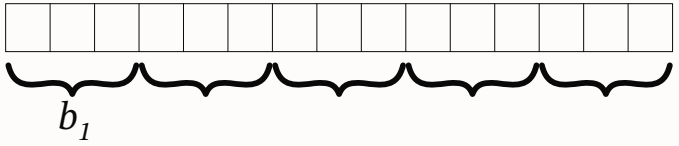
Secure Comparison

$Is\ \langle y \rangle_{\mathbb{Z}}{}^{R} + \langle y \rangle_{\mathbb{Z}}{}^{S} > n?$



Step 1: Equality Test of n-bit strings

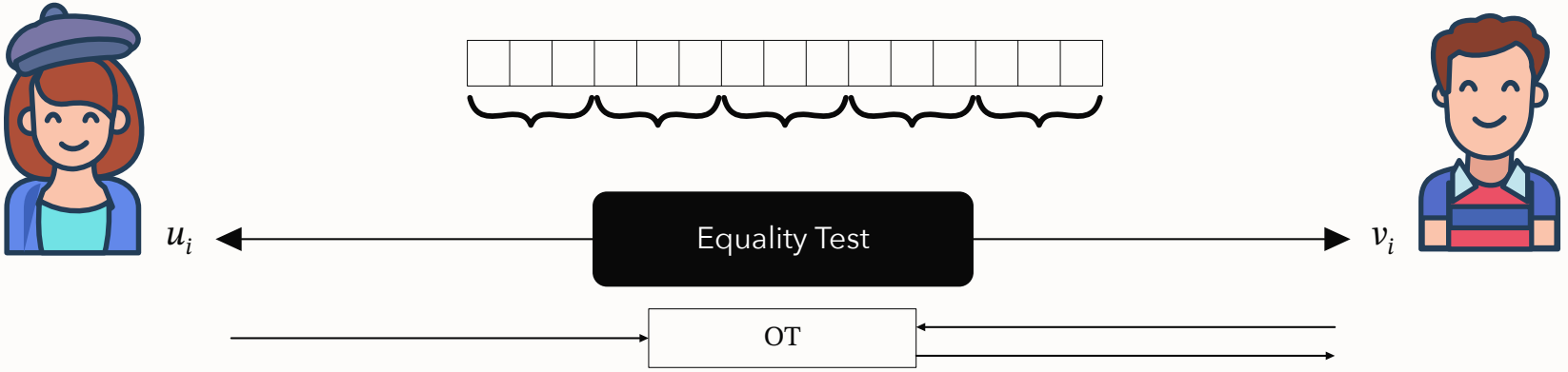




u_i ← Equality Test → v_i

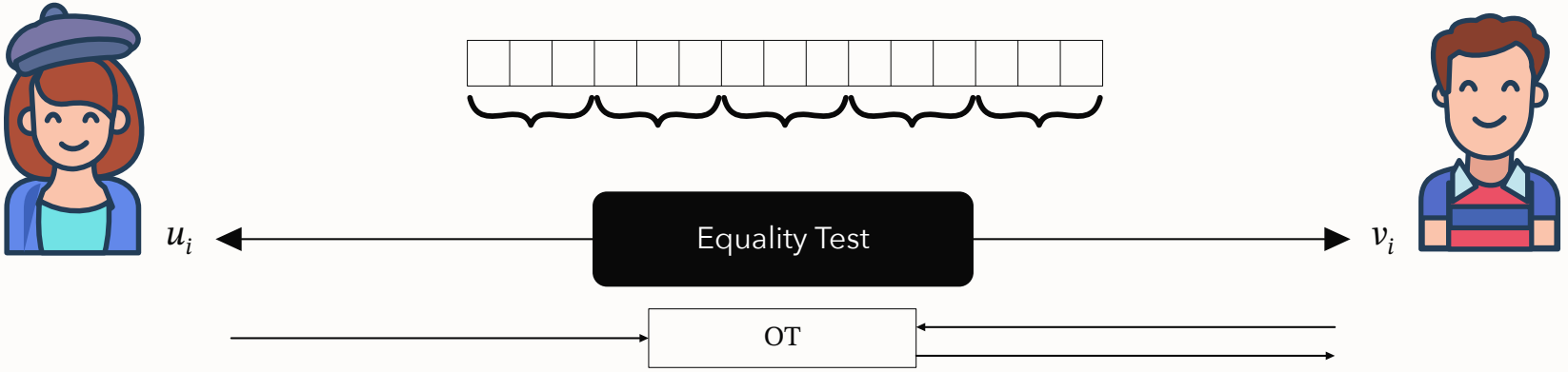
OT

Secure Comparison with Linear Communication and $O(log^* x)$ Rounds



Both parties get shares of 0 if $a_i = b_i$, and shares of 1 otherwise!

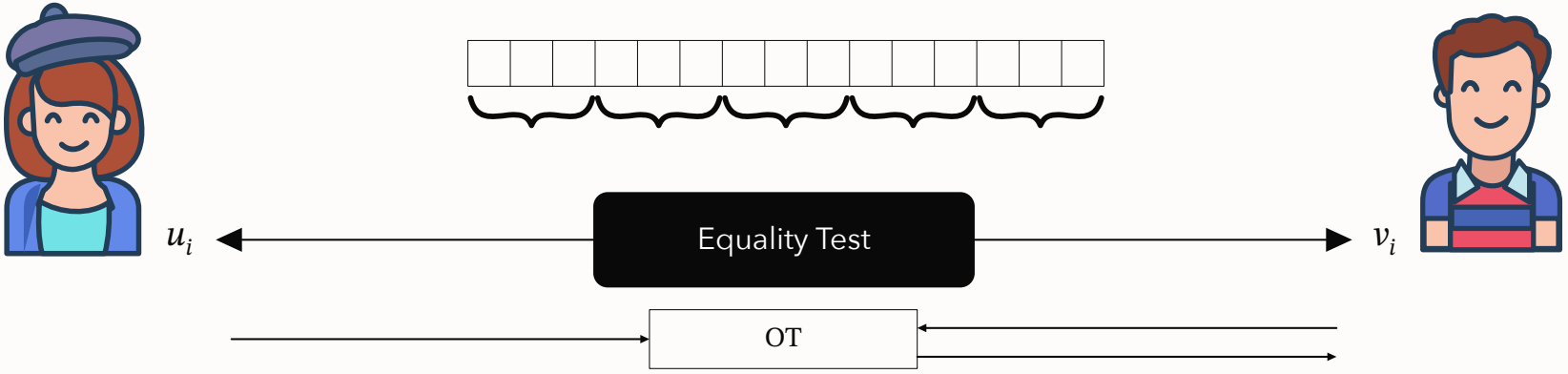
Secure Comparison with Linear Communication and $O(log^* x)$ Rounds



Simplified!

Both parties get shares of 0 if $a_i = b_i$, and shares of 1 otherwise!

Secure Comparison with Linear Communication and $O(\log^* x)$ Rounds



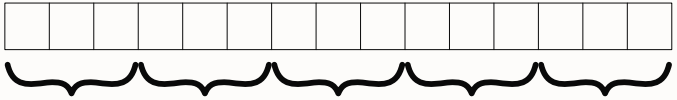
Simplified!

Both parties get shares of 0 if $a_i = b_i$, and shares of 1 otherwise!

→ Add them up over $\mathbb{Z}_n$, and check equality again (with a log n-bit equality test)

Secure Comparison with Linear Communication and $O(\log^* x)$ Rounds







Both parties get shares of 0 if $a_i = b_i$, and shares of 1 otherwise!

Secure Comparison with Linear Communication and $O(\log^* x)$ Rounds







Both parties get shares of 0 if $a_i = b_i$, and shares of 1 otherwise!

- Run Equality Test
- Find the first spot where they differ
- Compare just those strings

Secure Comparison with Linear Communication and $O(\log^* x)$ Rounds







Both parties get shares of 0 if $a_i = b_i$, and shares of 1 otherwise!

Keep running totals, run the checks in parallel, etc.

- Run Equality Test
- Find the first spot where they differ
- Compare just those strings

Conclusion:

OT with communication $O(n)$

and rounds $O(\log^* n)$

Conclusion







Communication: $O(n \cdot 2^{\log^* n})$

Rounds: $O((\log^* n)^2)$

Conclusion



OLE



Communication: $O(n \cdot 2^{\log^* n})$

Rounds: $O((\log^* n)^2)$



But… can't you do multiplication of n-bit integers with $O(n \log n)$ communication?

Conclusion



OLE



Communication: $O(n \cdot 2^{\log^* n})$
Rounds: $O((\log^* n)^2)$

But... can't you do multiplication of n-bit integers with $O(n \log n)$ communication?

YES! This is BETTER than evaluating the multiplication circuit in MPC.



And can you get malicious security?

Conclusion



OLE



Communication: $O(n \cdot 2^{\log^* n})$

Rounds: $O((\log^* n)^2)$



But... can't you do multiplication of n-bit integers with $O(n \log n)$ communication?

And can you get malicious security?

YES! This is BETTER than evaluating the multiplication circuit in MPC.

Yes, but using amortized, generic techniques.

Open Problems



- Can you do linear? This technique doesn't work.
- Can you do secure comparison in $O(n)$ communication and constant rounds? (wouldn't give the OLE an improvement even if you could.)
- Can you get a concrete improvement over the $O(n \log n)$ protocol? This isn't awful (gets better at fields of size $\approx$ 65,000 bits) but isn't good.
- Is there a better than generic way to get malicious security that has a nice proof?


That's all Folks!